DEVSECOPS LAB BOOK • PHASE 11

Chapter 31

Incident Response & Forensics

Security Incident Management

Detection & IOCs

Forensics Collection

Timeline Reconstruction

IR Playbooks

NIST-aligned incident response lifecycle

CHAPTER GOAL

Prepare Comprehensive Incident Response

Detect security incidents, contain threats, perform forensics, reconstruct attack timelines, and conduct post-incident analysis to prevent recurrence.

Detect

Identify security incidents via IOCs & tooling

Contain

Isolate affected systems, stop the bleeding

Investigate

Preserve evidence, perform digital forensics

Reconstruct

Build an attack timeline from logs

Recover

Restore clean systems, return to operations

Improve

Post-incident review, prevent recurrence

The NIST IR Cycle — Four Phases

PHASE 1
Preparation

- Tools & resources ready
- Procedures documented
- Team trained
- Comms plan established

PHASE 2
Detection & Analysis

- Monitor suspicious activity
- Identify incidents
- Classify severity & scope
- Activate IR team

PHASE 3
Containment, Eradication, Recovery

- Isolate affected systems
- Remove malware, close gaps
- Restore from clean backups
- Return to operations

PHASE 4
Post-Incident Activity

- Root cause analysis
- Document learnings
- Update procedures
- Implement preventive controls

Containment, Eradication & Recovery



Short-Term Containment

- Isolate affected systems
- Prevent further damage



Long-Term Containment

- Apply patches / configs
- Monitor for re-infection



Eradication

- Remove malware
- Close vulnerability
- Verify removal



Recovery

- Restore from clean backups
- Rebuild systems
- Return to operations

Indicators of Compromise (IOCs)

NETWORK-LEVEL

- Suspicious outbound connections
- Reverse shell / C&C traffic
- DNS tunneling
- SSL/TLS handshake anomalies

HOST-LEVEL

- Unexpected processes running
- New/modified SUID binaries
- Backdoor listeners
- Modified system files

APPLICATION-LEVEL

- SQL injection attempts
- Path traversal attacks
- Command injection / XXE
- Unusual error rates

LOG-BASED

- Multiple failed login attempts
- Privilege escalation attempts
- Access to sensitive files
- Firewall blocks

Where to Look, Tool by Tool

Tool	Detects	Example Signal
ELK / Elasticsearch	Auth failures aggregated by source IP	Multiple failed logins in 24h from one IP
Splunk	Privilege escalation patterns	sudo/su to root, count > 3 per user
Wazuh	Suspicious process execution (HIDS rule)	Execution from /tmp, rule level 7
Prometheus	Resource exhaustion abuse	CPU/memory spike over 5-minute window
tcpdump / network	Potential data exfiltration	> 10,000 ACK packets to one destination



Principle: no single tool sees the whole picture — correlate network, host, application, and log signals together before declaring an incident.

Digital Forensics Collection Checklist

Run in order — volatile evidence first, static evidence last

1. **Volatile memory**
   dd of physical memory / /proc capture
2. **Running processes**
   ps, pstree, lsof (network & all)
3. **Network state**
   netstat, ss, ip route/link, iptables
4. **System configuration**
   uname, hostnamectl, systemctl units
5. **User & group info**
   passwd, shadow, sudoers, last, w
6. **Filesystems & mounts**
   mount, df -h, lsblk
7. **Kernel modules**
   lsmod, /proc/modules
8. **Audit logs**
   audit.log, ausearch --ts recent
9. **System logs**
   /var/log archive, journalctl
10. **Filesystem snapshot**
    Full tar excluding proc/sys/dev/tmp

Evidence Integrity & Custody

Make evidence immutable

- chattr -R +i on the evidence directory
- chmod -R 000 to lock down access
- Never analyze the original — work from copies
- Hash every file (MD5/SHA) at collection time

Chain-of-custody log records

- Incident ID, date/time, and collecting system
- Collector identity and IP address of source host
- Evidence location and collection method
- MD5 hash of every collected file
- Every transfer: transferred by / received by / timestamp

From Raw Logs to a Reconstructed Timeline

1. Collect logs
2. Merge & sort by timestamp
3. **Find first IOC**
4. Map attack progression
5. Calculate duration

Sources merged into one timeline

- /var/log/auth.log — authentication & SSH
- /var/log/audit/audit.log — sudo & privilege events
- /var/log/syslog — errors, denials, alerts
- kubectl get events — cluster-level activity

Output: attack-timeline-<date>.txt — captures first IOC, minute-by-minute attack progression, recovery actions taken, and total attack duration (start → end).

Playbook: SQL Injection Attack

SEVERITY: HIGH *if database accessed*

INDICATORS

- Unusual SQL error messages in logs
- Large queries with "UNION SELECT"
- Database activity at odd hours
- Data exfiltration detected

HOW THIS FITS THE NIST CYCLE

- Detection: WAF / app logs surface the SQLi pattern above
- Containment: isolate the web server, activate blocking WAF rules
- Eradication: patch input validation, deploy parameterized queries
- Recovery: rotate DB credentials, verify data integrity, restore if needed

SQL Injection — Response Timeline

0–30 MIN
Immediate Actions

- Activate IR team & notify DBA
- Isolate affected web server
- Preserve logs & evidence
- Activate WAF blocking rules

30 MIN–2 HR
Investigation

- Identify source IP(s)
- Determine tables accessed
- Review app error logs
- Identify vulnerable endpoint

2–6 HR
Containment

- Input validation fix
- Parameterized queries
- Enable SQL query logging
- Patch & redeploy

6–24 HR
Recovery

- Verify fix effectiveness
- Rotate DB passwords/creds
- Enhanced logging enabled
- Verify data integrity

24–48 HR
Post-Incident

- Incident report & post-mortem
- Update security procedures
- Automated detection added
- Notify affected users

Incident Response Maturity Model

1

Reactive

Recovery: Weeks

- No preparation
- Ad-hoc response
- No procedures

2

Planned

Recovery: Days

- Basic procedures
- Response team identified
- Some tooling

3

Managed

Recovery: Hours

- Documented procedures
- Regular training
- Automated detection

4

Optimized

Recovery: Minutes

- Advanced playbooks
- Continuous monitoring
- Automation

KEY TAKEAWAY

Minutes, Not Weeks

Maturity is the difference between an incident that costs a weekend and one that costs a quarter.

WITHOUT A PLAYBOOK

- Response is improvised under pressure
- Evidence gets overwritten or lost
- No clear owner for each decision
- Root cause is never confirmed

WITH A PLAYBOOK

- Every phase has a checklist & owner
- Evidence is preserved and hashed
- Timeline reconstruction is routine
- Post-incident review closes the loop